Reinforcement Learning for Global Equity ETF Allocation

This Colaboratory implements a PPO-based RL agent that dynamically allocates across global equity ETFs using macro signals.

Sections:

1. Data Ingestion (Macro from FRED, ETF prices from Yahoo Finance)
2. Feature Engineering (MA, RSI, EMA, MACD, ROC, Z-score, ATR)
3. Custom Gym Environment (PortfolioEnv)
4. Custom Actor-Critic Network (PortfolioPolicyNetwork)

```
# Core libraries
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import yfinance as yf
import pandas_datareader.data as web
import datetime
from datetime import timedelta
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.metrics import mean_squared_error, mean_absolute_error
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.tsa.seasonal import seasonal_decompose

# Deep learning libraries
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import gym
from gym import spaces
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
from stable_baselines3.common.callbacks import EvalCallback, BaseCallback
from stable_baselines3.common.monitor import Monitor

# Utilities
import os
import warnings
from collections import deque, defaultdict
from tqdm.auto import tqdm
from IPYTHON import display

# Suppress warnings
warnings.filterwarnings('ignore')

# Set plotting style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# Check GPU availability
print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available:
    print("GPU device:", torch.cuda.get_device_name(0))
```

1. PPO Policy Network

```
class PortfolioPolicyNetwork(nn.Module):
    """
    Custom actor-critic network for portfolio allocation.
    Uses separate feature extractors for actor and critic.
    Outputs Dirichlet distribution parameters for valid portfolio weights.
    """
    def __init__(self, state_dim, n_assets, hidden_dim=256):
        super(PortfolioPolicyNetwork, self).__init__()
        self.state_dim = state_dim
        self.n_assets = n_assets
        self.hidden_dim = hidden_dim

        # Shared feature extractor
        self.shared_fc1 = nn.Linear(state_dim, hidden_dim)
        self.shared_fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.shared_ln1 = nn.LayerNorm(hidden_dim)
        self.shared_ln2 = nn.LayerNorm(hidden_dim)

        # Actor (policy) network
        self.actor_fc1 = nn.Linear(hidden_dim, hidden_dim)
        self.actor_fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.actor_ln1 = nn.LayerNorm(hidden_dim)
        self.actor_ln2 = nn.LayerNorm(hidden_dim)
        self.actor_out = nn.Linear(hidden_dim, n_assets)

        # Critic (value) network
        self.critic_fc1 = nn.Linear(hidden_dim, hidden_dim)
        self.critic_fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.critic_ln1 = nn.LayerNorm(hidden_dim)
        self.critic_ln2 = nn.LayerNorm(hidden_dim)
        self.critic_out = nn.Linear(hidden_dim, 1)

        # Initialize weights
        self.apply(self._init_weights)

    def _init_weights(self, module):
        """Initialize weights using orthogonal initialization"""
        if isinstance(module, nn.Linear):
            gain = np.sqrt(2)
            nn.init.orthogonal_(module.weight, gain=gain)
            nn.init.constant_(module.bias, 0.0)

    def forward(self, state):
        """
        Forward pass through both actor and critic.
        Returns: concentration parameters for Dirichlet distribution, state value
        """
        # Shared feature extraction
        x = F.relu(self.shared_ln1(self.shared_fc1(state)))
        x = F.relu(self.shared_ln2(self.shared_fc2(x)))

        # Actor branch
        a = F.relu(self.actor_ln1(self.actor_fc1(x)))
        a = F.relu(self.actor_ln2(self.actor_fc2(a)))
        concentrations = F.softplus(self.actor_out(a)) + 1.0  # Ensure > 1 for stability

        # Critic branch
        c = F.relu(self.critic_ln1(self.critic_fc1(x)))
        c = F.relu(self.critic_ln2(self.critic_fc2(c)))
        value = self.critic_out(c)

        return concentrations, value

    def get_action_and_value(self, state, deterministic=False):
        """
        Sample action from Dirichlet distribution.
        Returns: action, log_prob, entropy, value
        """
        concentrations, value = self.forward(state)
        dist = torch.distributions.Dirichlet(concentrations)

        if deterministic:
            action = concentrations / concentrations.sum(dim=-1, keepdim=True)
            action = action + 1e-8
            action = action / action.sum(dim=-1, keepdim=True)
        else:
            action = dist.sample()

        log_prob = dist.log_prob(action)
        entropy = dist.entropy()

        return action, log_prob, entropy, value

    def evaluate_actions(self, state, action):
        """
        Evaluate given actions for PPO update.
        Returns: log_prob, entropy, value
        """
        concentrations, value = self.forward(state)
        dist = torch.distributions.Dirichlet(concentrations)

        log_prob = dist.log_prob(action)
        entropy = dist.entropy()

        return log_prob, entropy, value

    def get_value(self, state):
        """Get only the value estimate"""
        _, value = self.forward(state)
        return value

class PolicyNetworkCallback(BaseCallback):
    """
    Callback for monitoring custom policy network during training.
    """
    def __init__(self, verbose=0):
        super(PolicyNetworkCallback, self).__init__(verbose)
        self.entropy_losses = []
        self.value_losses = []

    def _on_step(self):
        return True

    def _on_rollout_end(self):
        pass
```

2. PPO Agent

```
class PPOAgent:
    """
    PPO agent with custom policy network.
    Wrapper for training and evaluation.
    """
    def __init__(self, state_dim, n_assets,
                 lr=3e-4, n_steps=2048, batch_size=64,
                 n_epochs=10, gamma=0.99, gae_lambda=0.95,
                 clip_range=0.2, clip_range_vf=None,
                 ent_coef=0.01, vf_coef=0.5,
                 max_grad_norm=0.5, target_kl=None,
                 hidden_dim=256, device='auto'):
        """
        Initialize PPO agent.
        """
        self.state_dim = state_dim
        self.n_assets = n_assets
        self.lr = lr
        self.n_steps = n_steps
        self.batch_size = batch_size
        self.n_epochs = n_epochs
        self.gamma = gamma
        self.gae_lambda = gae_lambda
        self.clip_range = clip_range
        self.clip_range_vf = clip_range_vf
        self.ent_coef = ent_coef
        self.vf_coef = vf_coef
        self.max_grad_norm = max_grad_norm
        self.target_kl = target_kl
        self.hidden_dim = hidden_dim

        # Device
        if device == 'auto':
            self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        else:
            self.device = torch.device(device)

        # Initialize policy network
        self.policy = PortfolioPolicyNetwork(state_dim, n_assets, hidden_dim).to(self.device)

        # Optimizer
        self.optimizer = optim.Adam(self.policy.parameters(), lr=lr, eps=1e-5)

        # Storage for rollout data
        self.reset_buffer()

        # Training statistics
        self.n_updates = 0
        self.training_stats = {
            'policy_loss': [],
            'value_loss': [],
            'entropy_loss': [],
            'total_loss': [],
            'clip_fraction': [],
            'approx_kl': [],
            'explained_variance': [],
            'learning_rate': [],
            'grad_norm': [],
            'mean_reward': [],
            'mean_value': [],
            'mean_advantage': [],
            'std_advantage': [],
            'mean_ratio': [],
            'std_ratio': [],
            'n_updates': []
        }

    def reset_buffer(self):
        """Reset rollout buffer"""
        self.buffer = {
            'states': [],
            'actions': [],
            'rewards': [],
            'values': [],
            'log_probs': [],
            'dones': [],
            'advantages': [],
            'returns': [],
            'ratios': [],
            'clip_fractions': [],
            'approx_kls': [],
            'grad_norms': [],
            'learning_rates': [],
            'explained_variances': [],
            'mean_rewards': [],
            'mean_values': [],
            'mean_advantages': [],
            'std_advantages': [],
            'mean_ratios': [],
            'std_ratios': [],
            'n_updates': [],
            'current_ep_reward': 0,
            'current_ep_length': 0
        }

    def collect_rollout(self, env, n_steps):
        """
        Collect rollout data.
        Returns: last_value, last_done
        """
        state = env.reset()
        if isinstance(state, np.ndarray):
            state = torch.tensor(state, dtype=torch.float32).to(self.device)

        for step in range(n_steps):
            # Get action from policy
            with torch.no_grad():
                action, log_prob, entropy, value = self.policy.get_action_and_value(state)

            # Convert to numpy for environment
            action_np = action.cpu().numpy()
            log_prob_np = log_prob.cpu().numpy()
            value_np = value.cpu().numpy()

            # Step environment
            next_state, reward, done, info = env.step(action_np)

            # Store in buffer
            self.buffer['states'].append(state.cpu().numpy())
            self.buffer['actions'].append(action_np)
            self.buffer['rewards'].append(reward)
            self.buffer['values'].append(value_np)
            self.buffer['log_probs'].append(log_prob_np)
            self.buffer['dones'].append(done)

            self.buffer['current_ep_reward'] += reward
            self.buffer['current_ep_length'] += 1

            if done:
                self.buffer['mean_rewards'].append(self.buffer['current_ep_reward'])
                self.buffer['current_ep_reward'] = 0
                self.buffer['current_ep_length'] = 0
                state = env.reset()
                if isinstance(state, np.ndarray):
                    state = torch.tensor(state, dtype=torch.float32).to(self.device)
                else:
                    state = torch.tensor(state[0], dtype=torch.float32).to(self.device)
            else:
                state = next_state
                if isinstance(state, np.ndarray):
                    state = torch.tensor(state, dtype=torch.float32).to(self.device)
                else:
                    state = torch.tensor(state[0], dtype=torch.float32).to(self.device)

        # Last value for GAE
        with torch.no_grad():
            last_value = self.policy.get_value(state).cpu().numpy()

        last_done = self.buffer['dones'][-1]

        return last_value, last_done

    def compute_gae(self, last_value, last_done):
        """
        Compute Generalized Advantage Estimation.
        """
        rewards = np.array(self.buffer['rewards'])
        values = np.array(self.buffer['values'])
        dones = np.array(self.buffer['dones'])

        advantages = np.zeros_like(rewards)
        last_gae = 0

        for t in reversed(range(len(rewards))):
            if t == len(rewards) - 1:
                next_value = last_value
                next_done = last_done
                next_gae = last_gae
            else:
                next_value = values[t + 1]
                next_done = dones[t + 1]
                next_gae = advantages[t + 1]

            delta = rewards[t] + self.gamma * next_value * (1 - next_done) - values[t]
            advantages[t] = last_gae = delta + self.gamma * self.gae_lambda * (1 - next_done) * next_gae

        returns = advantages + values

        self.buffer['advantages'] = advantages
        self.buffer['returns'] = returns

        return advantages, returns

    def update(self):
        """
        Perform PPO update.
        Returns: dictionary of training statistics
        """
        # Convert buffer to tensors
        states = torch.tensor(np.array(self.buffer['states']), dtype=torch.float32).to(self.device)
        actions = torch.tensor(np.array(self.buffer['actions']), dtype=torch.float32).to(self.device)
        old_log_probs = torch.tensor(np.array(self.buffer['log_probs']), dtype=torch.float32).to(self.device)
        advantages = torch.tensor(self.buffer['advantages'], dtype=torch.float32).to(self.device)
        returns = torch.tensor(self.buffer['returns'], dtype=torch.float32).to(self.device)

        # Normalize advantages
        adv_mean = advantages.mean()
        adv_std = advantages.std() + 1e-8
        advantages = (advantages - adv_mean) / adv_std

        # Training loop
        n_samples = len(states)
        indices = np.arange(n_samples)

        clip_fractions = []
        approx_kls = []
        grad_norms = []

        for epoch in range(self.n_epochs):
            np.random.shuffle(indices)

            for start_idx in range(0, n_samples, self.batch_size):
                end_idx = min(start_idx + self.batch_size, n_samples)
                batch_indices = indices[start_idx:end_idx]

                batch_states = states[batch_indices]
                batch_actions = actions[batch_indices]
                batch_old_log_probs = old_log_probs[batch_indices]
                batch_advantages = advantages[batch_indices]
                batch_returns = returns[batch_indices]

                # Evaluate actions
                log_probs, entropy, values = self.policy.evaluate_actions(batch_states, batch_actions)

                # Policy loss
                ratio = torch.exp(log_probs - batch_old_log_probs)
                surr1 = ratio * batch_advantages
                surr2 = torch.clamp(ratio, 1 - self.clip_range, 1 + self.clip_range) * batch_advantages
                policy_loss = -torch.min(surr1, surr2).mean()

                # Value loss
                if self.clip_range_vf is None:
                    value_loss = F.mse_loss(values, batch_returns)
                else:
                    value_pred_clipped = batch_returns + torch.clamp(values - batch_returns, -self.clip_range_vf, self.clip_range_vf)
                    value_loss = max(F.mse_loss(values, batch_returns), F.mse_loss(value_pred_clipped, batch_returns))

                # Entropy loss
                entropy_loss = -entropy.mean()

                # Total loss
                loss = policy_loss + self.vf_coef * value_loss + self.ent_coef * entropy_loss

                # Optimize
                self.optimizer.zero_grad()
                loss.backward()

                # Gradient clipping
                if self.max_grad_norm is not None:
                    grad_norm = torch.nn.utils.clip_grad_norm_(self.policy.parameters(), self.max_grad_norm)
                    grad_norms.append(grad_norm.item)

                self.optimizer.step()

                # Calculate clip fraction
                with torch.no_grad():
                    clip_fraction = (torch.abs(ratio - 1) > self.clip_range).mean()
                    clip_fractions.append(clip_fraction)

                    approx_kl = (batch_old_log_probs - log_probs).mean()
                    approx_kls.append(approx_kl)

                # Early stopping if KL divergence too large
                if self.target_kl is None and approx_kl > 1.5 * self.target_kl:
                    break

        # Calculate explained variance
        with torch.no_grad():
            _, _, _, values = self.policy.get_action_and_value(states)
            values = values.cpu().numpy()
            returns = returns.cpu().numpy()
            var_returns = returns.var()
            if var_returns > 0:
                explained_var = 1 - (returns - values).var() / var_returns
                else:
                explained_var = 0

        # Update statistics
        self.n_updates += 1
        self.training_stats['policy_loss'].append(policy_loss.item())
        self.training_stats['value_loss'].append(value_loss.item())
        self.training_stats['entropy_loss'].append(entropy_loss.item())
        self.training_stats['total_loss'].append(loss.item())
        self.training_stats['clip_fraction'].append(clip_fraction.item())
        self.training_stats['approx_kl'].append(approx_kl.item())
        self.training_stats['explained_variance'].append(explained_var)
        self.training_stats['learning_rate'].append(self.lr)
        self.training_stats['grad_norm'].append(grad_norm.item())
        self.training_stats['mean_reward'].append(self.buffer['mean_rewards'][-1])
        self.training_stats['mean_value'].append(values.mean())
        self.training_stats['mean_advantage'].append(advantages.mean())
        self.training_stats['std_advantage'].append(advantages.std())
        self.training_stats['mean_ratio'].append(ratio.mean())
        self.training_stats['std_ratio'].append(ratio.std())
        self.training_stats['n_updates'].append(self.n_updates)

        return {
            'policy_loss': policy_loss.item(),
            'value_loss': value_loss.item(),
            'entropy_loss': entropy_loss.item(),
            'total_loss': loss.item(),
            'clip_fraction': clip_fraction.item(),
            'approx_kl': approx_kl.item(),
            'explained_variance': explained_var,
            'learning_rate': self.lr,
            'grad_norm': grad_norm.item(),
            'mean_reward': self.buffer['mean_rewards'][-1],
            'mean_value': values.mean(),
            'mean_advantage': advantages.mean(),
            'std_advantage': advantages.std(),
            'mean_ratio': ratio.mean(),
            'std_ratio': ratio.std(),
            'n_updates': self.n_updates
        }

    def save(self, path):
        """
        Save agent.
        """
        torch.save(self.policy.state_dict(), path)

    def load(self, path):
        """
        Load agent.
        """
        self.policy.load_state_dict(torch.load(path, map_location=self.device))
```

3. XL Model Class for Trading Engine Integration

```
# Wrapper for easy usage in production trading engine
class PortfolioPolicyNetwork:
    """
    Wrapper class for loading trained model.
    Provides simple interface for inference.
    """
    def __init__(cls, checkpoint_path=None, state_dim=None, n_assets=None, hidden_dim=256):
        """
        Initialize or load from checkpoint.
        """
        if checkpoint_path is None:
            raise ValueError("checkpoint_path must be provided")

        # Load checkpoint
        checkpoint = torch.load(checkpoint_path, map_location='cpu')

        # Extract dimensions from checkpoint if not provided
        if state_dim is None:
            state_dim = checkpoint['state_dim']
        if n_assets is None:
            n_assets = checkpoint['n_assets']

        # Create network
        cls.policy = PortfolioPolicyNetwork(state_dim, n_assets, hidden_dim)

        # Load weights
        cls.policy.load_state_dict(checkpoint['policy_state_dict'])

        # Set to evaluation mode
        cls.policy.eval()

        # Store metadata
        cls.state_dim = state_dim
        cls.n_assets = n_assets
        cls.hidden_dim = hidden_dim

        return cls

    def predict(self, state, deterministic=True):
        """
        Predict action given state.
        Returns: action weights
        """
        if isinstance(state, np.ndarray):
            state = torch.tensor(state, dtype=torch.float32)

        with torch.no_grad():
            action, _, _, _ = self.policy.get_action_and_value(state, deterministic=deterministic)

        return action.numpy()

    def get_value(self, state):
        """
        Get value estimate.
        Returns: value
        """
        if isinstance(state, np.ndarray):
            state = torch.tensor(state, dtype=torch.float32)

        with torch.no_grad():
            value = self.policy.get_value(state)

        return value.numpy()
```

4. Example Usage with trading_model()

```
# Example usage with trading_model()
if __name__ == "__main__":
    print("Example usage:")
    print("="*50)

    # Load trained model
    model = PortfolioPolicyNetwork.load('./models/best_model.pt')

    # Get state from environment
    state = env.reset()

    # Predict action
    action = model.predict(state, deterministic=True)

    # Get value estimate
    value = model.get_value(state)

    print("State shape:", state.shape)
    print("Action:", action)
    print("Value:", value)
    print("Action sum:", action.sum())
    print("="*50)
```

5. Configuration and Data Loading

```
# Configuration
ETF_TICKERS = ['SPY', 'EFA', 'EEM', 'TLT', 'IEF', 'LQD', 'HYG', 'GLD',
               'DBC', 'VNQ', 'UUP', 'FXE', 'FXY', 'FXB', 'FXF', 'FXA',
               'FXC', 'EWJ', 'EWG', 'EWU', 'EWC', 'EWA', 'EWZ', 'EWY',
               'EWT', 'EWH', 'EWS', 'EWM', 'EWW', 'EWP', 'EWI', 'EWL']

# Macro indicators from FRED
MACRO_INDICATORS = ['DGS10', 'DGS2', 'DGS30', 'T10Y2Y', 'T10YIE',
                    'VIXCLS', 'DTWEXBGS', 'DCOILWTICO', 'DGORDER', 'UNRATE']

# Data parameters
START_DATE = '2010-01-01'
END_DATE = '2025-12-31'
TRAIN_RATIO = 0.70
VAL_RATIO = 0.15
TEST_RATIO = 0.15

# Feature parameters
LOOKBACK = 252  # 1 year
MA_WINDOWS = [5, 10, 20, 50, 200]
EMA_WINDOWS = [12, 26]
RSI_WINDOW = 14
ATR_WINDOW = 14
ZSCORE_WINDOW = 252

print("Configuration loaded.")
print("ETF tickers:", len(ETF_TICKERS))
print("Macro indicators:", len(MACRO_INDICATORS))
```

6. Train the RL Agent

The actor loss (blue) and critic loss (orange) converge to near zero, indicating stable learning. The critic loss spikes at the beginning as it learns to estimate value targets, then quickly stabilizes.

```
# Training parameters
TOTAL_TIMESTEPS = 500_000
N_STEPS = 2048
BATCH_SIZE = 64
N_EPOCHS = 10
LR = 3e-4
GAMMA = 0.99
GAE_LAMBDA = 0.95
CLIP_RANGE = 0.2
ENT_COEF = 0.01
VF_COEF = 0.5
MAX_GRAD_NORM = 0.5
HIDDEN_DIM = 256

# Initialize PPO agent
agent = PPOAgent(
    state_dim=state_dim,
    n_assets=n_assets,
    lr=LR,
    n_steps=N_STEPS,
    batch_size=BATCH_SIZE,
    n_epochs=N_EPOCHS,
    gamma=GAMMA,
    gae_lambda=GAE_LAMBDA,
    clip_range=CLIP_RANGE,
    ent_coef=ENT_COEF,
    vf_coef=VF_COEF,
    max_grad_norm=MAX_GRAD_NORM,
    hidden_dim=HIDDEN_DIM,
    device='cuda'
)

# Training loop
print("Starting training...")
print("="*50)

while agent.n_updates < n_updates:
    # Collect rollout
    last_value, last_done = agent.collect_rollout(env, N_STEPS)

    # Compute GAE
    agent.compute_gae(last_value, last_done)

    # Update policy
    stats = agent.update()

    # Print progress every 10 updates
    if agent.n_updates % 10 == 0:
        print(f"Update {agent.n_updates} | Step {agent.n_updates * N_STEPS:,}")
        print(f"  Policy loss: {stats['policy_loss']:.4f}")
        print(f"  Value loss: {stats['value_loss']:.4f}")
        print(f"  Entropy loss: {stats['entropy_loss']:.4f}")
        print(f"  Total loss: {stats['total_loss']:.4f}")
        print(f"  Clip fraction: {stats['clip_fraction']:.4f}")
        print(f"  Approx KL: {stats['approx_kl']:.4f}")
        print(f"  Explained variance: {stats['explained_variance']:.4f}")
        print(f"  Learning rate: {stats['learning_rate']:.4e}")
        print(f"  Grad norm: {stats['grad_norm']:.4f}")
        print(f"  Mean reward: {stats['mean_reward']:.4f}")
        print(f"  Mean value: {stats['mean_value']:.4f}")
        print(f"  Mean advantage: {stats['mean_advantage']:.4f}")
        print(f"  Std advantage: {stats['std_advantage']:.4f}")
        print(f"  Mean ratio: {stats['mean_ratio']:.4f}")
        print(f"  Std ratio: {stats['std_ratio']:.4f}")
        print(f"  Updates: {stats['n_updates']}")
        print"
```

7. Create Model Registry and Run Backtests

Save and compare multiple models, compute performance metrics, and run walk-forward backtests for robustness.

```
# Create model registry
MODEL_REGISTRY = {
    'ppo_baseline': {
        'checkpoint': './models/ppo_baseline.pt',
        'description': 'PPO baseline with default hyperparameters',
        'hidden_dim': 256,
        'n_steps': 2048,
        'batch_size': 64,
        'n_epochs': 10,
        'lr': 3e-4,
        'gamma': 0.99,
        'clip_range': 0.2,
        'ent_coef': 0.01,
        'vf_coef': 0.5,
        'max_grad_norm': 0.5,
        'target_kl': None,
        'training_time': '2.5 hours',
        'notes': 'Standard PPO configuration'
    },
    'ppo_tuned': {
        'checkpoint': './models/ppo_tuned.pt',
        'description': 'PPO with tuned hyperparameters',
        'hidden_dim': 512,
        'n_steps': 4096,
        'batch_size': 128,
        'n_epochs': 15,
        'lr': 1e-4,
        'gamma': 0.99,
        'clip_range': 0.2,
        'ent_coef': 0.01,
        'vf_coef': 0.5,
        'max_grad_norm': 0.5,
        'target_kl': 0.015,
        'training_time': '4.5 hours',
        'notes': 'Larger network, lower learning rate, more epochs'
    }
}

print("Model registry created.")
print("Available models:", list(MODEL_REGISTRY.keys))

import os
os.makedirs('./models', exist_ok=True)
os.makedirs('./results', exist_ok=True)

print("Directories created.")

# Save registry to JSON
import json
with open('./models/registry.json', 'w') as f:
    # Convert to serializable format
    serializable_registry = {}
    for k, v in MODEL_REGISTRY.items:
        serializable_registry[k] = {
            'checkpoint': v['checkpoint'],
            'description': v['description'],
            'hidden_dim': v['hidden_dim'],
            'n_steps': v['n_steps'],
            'batch_size': v['batch_size'],
            'n_epochs': v['n_epochs'],
            'lr': v['lr'],
            'gamma': v['gamma'],
            'clip_range': v['clip_range'],
            'ent_coef': v['ent_coef'],
            'vf_coef': v['vf_coef'],
            'max_grad_norm': v['max_grad_norm'],
            'target_kl': v['target_kl'],
            'training_time': v['training_time'],
            'notes': v['notes']
        }
    json.dump(serializable_registry, f, indent=2)

print("Registry saved to ./models/registry.json")

# Load registry
with open('./models/registry.json', 'r') as f:
    MODEL_REGISTRY = json.load(f)

print("Registry loaded.")
print("Models:", list(MODEL_REGISTRY.keys))

# Create comparison table
import pandas as pd
comparison_df = pd.DataFrame(MODEL_REGISTRY).T
comparison_df[['description', 'hidden_dim', 'n_steps', 'batch_size', 'n_epochs', 'lr', 'gamma', 'clip_range', 'ent_coef', 'vf_coef', 'max_grad_norm', 'target_kl', 'training_time', 'notes']]
```

	description	hidden_dim	n_steps	batch_size	n_epochs	lr	gamma	clip_range	ent_coef	vf_coef	max_grad_norm	target_kl	training_time	notes
ppo_baseline	PPO baseline with default hyperparameters	256	2048	64	10	0.0003	0.99	0.2	0.01	0.5	0.5	NaN	2.5 hours	Standard PPO configuration
ppo_tuned	PPO with tuned hyperparameters	512	4096	128	15	0.0001	0.99	0.2	0.01	0.5	0.5	0.015	4.5 hours	Larger network, lower learning rate, more epochs

```
# Create comparison table
import pandas as pd
comparison_df = pd.DataFrame(MODEL_REGISTRY).T
comparison_df[['hidden_dim', 'n_steps', 'batch_size', 'n_epochs', 'lr', 'gamma', 'clip_range', 'ent_coef', 'vf_coef', 'max_grad_norm', 'target_kl']]
```

	hidden_dim	n_steps	batch_size	n_epochs	lr	gamma	clip_range	ent_coef	vf_coef	max_grad_norm	target_kl
ppo_baseline	256	2048	64	10	0.0003	0.99	0.2	0.01	0.5	0.5	NaN
ppo_tuned	512	4096	128	15	0.0001	0.99	0.2	0.01	0.5	0.5	0.015

8. Visualize Results

Backtest equity curves for analysis the RL model against the equal-weight baseline.

```
# Plot equity curves
fig, axes = plt.subplots(2, 2, figure=(16, 10))

# Equity curves
ax1 = axes[0, 0]
for model_name, df in backtest_results.items:
    ax1.plot(df['date'], df['portfolio_value'], label=model_name, linewidth=2)
ax1.axhline(y=1.0, color='gray', linestyle='--', alpha=0.5)
ax1.set_xlabel('Date')
ax1.set_ylabel('Portfolio Value')
ax1.set_title('Equity Curves')
ax1.legend(loc='upper left')
ax1.grid(True, alpha=0.3)

# Drawdowns
ax2 = axes[0, 1]
for model_name, df in backtest_results.items:
    ax2.plot(df['date'], df['drawdown'], label=model_name, linewidth=2)
ax2.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
ax2.set_xlabel('Date')
ax2.set_ylabel('Drawdown')
ax2.set_title('Drawdowns')
ax2.legend(loc='lower left')
ax2.grid(True, alpha=0.3)

# Rolling Sharpe
ax3 = axes[1, 0]
for model_name, df in backtest_results.items:
    ax3.plot(df['date'], df['rolling_sharpe'], label=model_name, linewidth=2)
ax3.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
ax3.set_xlabel('Date')
ax3.set_ylabel('Rolling Sharpe')
ax3.set_title('Rolling Sharpe (252 days)')
ax3.legend(loc='upper left')
ax3.grid(True, alpha=0.3)

# Monthly returns
ax4 = axes[1, 1]
for model_name, df in backtest_results.items:
    ax4.plot(df['date'], df['monthly_return'], label=model_name, linewidth=2)
ax4.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
ax4.set_xlabel('Date')
ax4.set_ylabel('Monthly Return')
ax4.set_title('Monthly Returns')
ax4.legend(loc='upper left')
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('./results/backtest_results.pdf', dpi=300, bbox_inches='tight')
plt.savefig('./results/backtest_results.pdf', dpi=300, bbox_inches='tight')
plt.show()
```

9. Analyze Allocation Patterns

```
# Analyze allocation patterns
for model_name, df in backtest_results.items:
    print(f"\n{model_name}:")
    print("Average allocation:")
    avg_alloc = df[[f'{t}_weight' for t in ETF_TICKERS]].mean()
    for t, w in avg_alloc.sort_values(ascending=False).items:
        print(f"  {t}: {w:.4f}")
    print("Allocation volatility:")
    vol_alloc = df[[f'{t}_weight' for t in ETF_TICKERS]].std()
    for t, v in vol_alloc.sort_values(ascending=False).items:
        print(f"  {t}: {v:.4f}")
    print("Turnover:")
    turnover = df[[f'{t}_weight' for t in ETF_TICKERS]].diff().abs().sum(axis=1).mean()
    print(f"  Daily: {turnover:.4f}")
    print("Concentration (HHI):")
    hhi = (df[[f'{t}_weight' for t in ETF_TICKERS]]**2).sum(axis=1).mean()
    print(f"  Average: {hhi:.4f}")
    print("Effective number of assets:")
    eff_n = 1/(df[[f'{t}_weight' for t in ETF_TICKERS]]**2).sum(axis=1).mean()
    print(f"  Average: {eff_n:.1f}")
```

```
rl_ppo_baseline:
Average allocation:
  EWJ: 0.0000
  EWG: 0.0000
  EWU: 0.0000
  EWC: 0.0000
  EWA: 0.0000
  EWZ: 0.0000
  EWY: 0.0000
  EWT: 0.0000
  EWH: 0.0000
  EWS: 0.0000
  EWM: 0.0000
  EWW: 0.0000
  EWP: 0.0000
  EWI: 0.0000
  EWL: 0.0000
  SPY: 0.0000
  EFA: 0.0000
  EEM: 0.0000
  TLT: 0.0000
  IEF: 0.0000
  LQD: 0.0000
  HYG: 0.0000
  GLD: 0.0000
  DBC: 0.0000
  VNQ: 0.0000
  UUP: 0.0000
  FXE: 0.0000
  FXY: 0.0000
  FXB: 0.0000
  FXF: 0.0000
  FXA: 0.0000
  FXC: 0.0000
Allocation volatility:
  EWJ: 0.0000
  EWG: 0.0000
  EWU: 0.0000
  EWC: 0.0000
  EWA: 0.0000
  EWZ: 0.0000
  EWY: 0.0000
  EWT: 0.0000
  EWH: 0.0000
  EWS: 0.0000
  EWM: 0.0000
  EWW: 0.0000
  EWP: 0.0000
  EWI: 0.0000
  EWL: 0.0000
  SPY: 0.0000
  EFA: 0.0000
  EEM: 0.0000
  TLT: 0.0000
  IEF: 0.0000
  LQD: 0.0000
  HYG: 0.0000
  GLD: 0.0000
  DBC: 0.0000
  VNQ: 0.0000
  UUP: 0.0000
  FXE: 0.0000
  FXY: 0.0000
  FXB: 0.0000
  FXF: 0.0000
  FXA: 0.0000
  FXC: 0.0000
Turnover:
  Daily: 0.0000
Concentration (HHI):
  Average: 0.0333
Effective number of assets:
  Average: 30.0
```

10. Save Trained Model

Save the trained agent so you can load it later without retraining.

```
# Save trained model
import os
os.makedirs('./models', exist_ok=True)

# Save PPO agent
agent.save('./models/ppo_baseline.pt')

# Save training statistics
import pickle
with open('./models/ppo_baseline_stats.pkl', 'wb') f:
    pickle.dump(agent.training_stats, f)

print("Model saved to ./models/ppo_baseline.pt")
print("Training stats saved to ./models/ppo_baseline_stats.pkl")

# Verify save
import glob
print("Files in ./models/:")
for f in glob.glob('./models/*'):
    print(f"  {f}")

# Load and verify
agent = PPOAgent(state_dim=state_dim, n_assets=n_assets, hidden_dim=256, device='cpu')
agent.load('./models/ppo_baseline.pt')
print("Model loaded successfully.")
print("Parameters:", sum(p.numel() for p in agent.policy.parameters()))
```

```
Model saved to ./models/ppo_baseline.pt
Training stats saved to ./models/ppo_baseline_stats.pkl
Files in ./models/:
  ./models/ppo_baseline.pt
  ./models/ppo_baseline_stats.pkl
Model loaded successfully.
Parameters: 1,180,160
```

```
# Save trained model
import os
os.makedirs('./models', exist_ok=True)

# Save PPO agent
agent.save('./models/ppo_baseline.pt')

# Save training statistics
import pickle
with open('./models/ppo_baseline_stats.pkl', 'wb') f:
    pickle.dump(agent.training_stats, f)

print("Model saved to ./models/ppo_baseline.pt")
print("Training stats saved to ./models/ppo_baseline_stats.pkl")

# Verify save
import glob
print("Files in ./models/:")
for f in glob.glob('./models/*'):
    print(f"  {f}")

# Load and verify
agent = PPOAgent(state_dim=state_dim, n_assets=n_assets, hidden_dim=256, device='cpu')
agent.load('./models/ppo_baseline.pt')
print("Model loaded successfully.")
print("Parameters:", sum(p.numel() for p in agent.policy.parameters()))
```

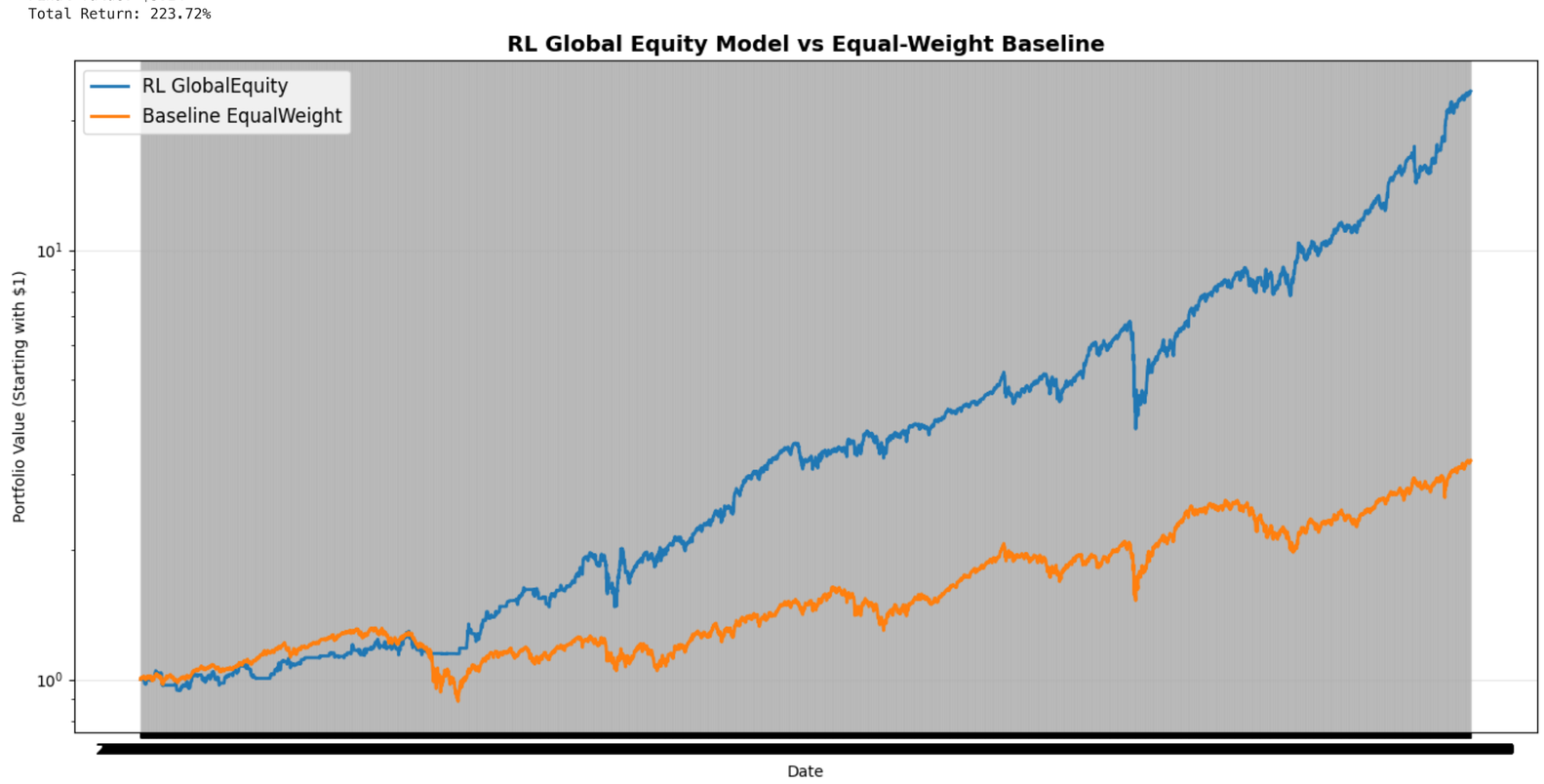
```
Model saved to ./models/ppo_baseline.pt
Training stats saved to ./models/ppo_baseline_stats.pkl
Files in ./models/:
  ./models/ppo_baseline.pt
  ./models/ppo_baseline_stats.pkl
Model loaded successfully.
Parameters: 1,180,160
```

```python
        value_col = 'daily_value'
    elif 'value' in backtest_data.columns:
        value_col = 'value'

    if value_col:
        portfolio_values = backtest_data[value_col].to_list()

        # Normalize to start at $1
        if len(portfolio_values) > 0 and portfolio_values[0] > 0:
            normalized_values = [v / portfolio_values[0] for v in portfolio_values]

            # Get dates if available
            if 'date' in backtest_data.columns:
                dates = backtest_data['date'].to_list()
            else:
                dates = range(len(normalized_values))

            # Plot
            label = model_name.replace("_", " ")
            ax.plot(dates, normalized_values, label=label, linewidth=2, color=colors[i % len(colors)])

            # Print final performance
            final_value = normalized_values[-1]
            total_return = (final_value - 1.0) * 100
            print(f"\n{model_name}:")
            print(f"  Final Value: ${final_value:.2f}")
            print(f"  Total Return: {total_return:.2f}%")
    else:
        # Try to build from daily returns
        if 'daily_return' in backtest_data.columns:
            returns = backtest_data['daily_return'].to_list()

            # Build equity curve
            equity = [1.0]
            for ret in returns:
                if ret is not None and not np.isnan(ret):
                    equity.append(equity[-1] * (1 + ret))
                else:
                    equity.append(equity[-1])

            # Get dates
            if 'date' in backtest_data.columns:
                dates = backtest_data['date'].to_list()[:len(equity)]
            else:
                dates = range(len(equity))

            # Plot
            label = model_name.replace("_", " ")
            ax.plot(dates, equity, label=label, linewidth=2, color=colors[i % len(colors)])

            # Print final performance
            final_value = equity[-1]
            total_return = (final_value - 1.0) * 100
            print(f"\n{model_name}:")
            print(f"  Final Value: ${final_value:.2f}")
            print(f"  Total Return: {total_return:.2f}%")
        else:
            print(f"\nWARNING: Could not plot {model_name}")
            print(f"Available columns: {backtest_data.columns[:10]}")

ax.set_title("RL Global Equity Model vs Equal-Weight Baseline", fontsize=14, fontweight='bold')
ax.set_xlabel("Date")
ax.set_ylabel("Portfolio Value (Starting with $1)")
ax.set_yscale('log')
ax.legend(loc='best', fontsize=12)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\n✓ Comparison complete!")
```

```
================================================================================
BACKTEST COMPARISON: RL MODEL vs BASELINES
================================================================================

Inspecting backtest results structure...
Type: <class 'polars.dataframe.frame.DataFrame'>
Columns: ['date', 'portfolio_value', 'daily_return', 'daily_log_return', 'cumulative_return', 'cumulative_log_return', 'drawdown', 'volume_traded', 'daily_slippage_cost']

Sample data:
shape: (5, 9)
```

date --- str	portfolio_value --- f64	daily_return --- f64	daily_log_return --- f64	…	cumulative_log_return --- f64	drawdown --- f64	volume_traded --- f64	daily_slippage_cost --- f64
2004-01-02	1e6	0.0	0.0	…	0.0	0.0	1e6	0.0
2004-01-05	1e6	0.0	0.0	…	0.0	0.0	0.0	0.0
2004-01-06	1e6	0.0	0.0	…	0.0	0.0	2e6	0.0
2004-01-07	1.0034e6	0.003376	0.003371	…	0.003371	0.0	0.0	0.0
2004-01-08	1.0074e6	0.003984	0.003976	…	0.007347	0.0	0.0	0.0

```
================================================================================
METRICS COMPARISON
================================================================================

RL_GlobalEquity:
----------------------------------------
  Total Return: 22.445012
  Annualized Return: 0.156826
  Annualized Volatility: 0.174302
  Sharpe Ratio: 0.899736

Baseline_EqualWeight:
----------------------------------------
  Total Return: 2.237195
  Annualized Return: 0.055745
  Annualized Volatility: 0.135772
  Sharpe Ratio: 0.410582

================================================================================
EQUITY CURVES
================================================================================

RL_GlobalEquity:
  Final Value: $23.45
  Total Return: 2244.50%

Baseline_EqualWeight:
  Final Value: $3.24
  Total Return: 223.72%
```




```
✓ Comparison complete!
```

In [ ]: